

|    |            |    |  |
|----|------------|----|--|
| 学号 | 1906410229 | 成绩 |  |
|----|------------|----|--|



沈阳建筑大学

# 课程设计说明书

设计名称 综合课程设计 1

设计题目 java 商品秒杀设计

设计时间 2022 年 5 月 30 日—6 月 17 日

学 院 电气与控制工程学院

专 业 计算机科学与技术

班 级 计算机 1902 班

姓 名 蔡艺洵

指导教师 曹阳 王守金 韩子扬 郑新录 谢军

2022 年 6 月 16 日

# 综合课程设计说明书

## 目 录

- 一、 题目简介
- 二、 功能说明
- 三、 程序框图
- 四、 程序清单
- 五、 运行结果
- 六、 设计体会
- 七、 参考文献
- 八、 教师评语

## 一、项目简介

针对于类似淘宝、天猫、京东等大型电商平台，总会有一些促销活动，而最常见的就是抢购模式。在抢购模式中，有一类极难处理的情况，那就是秒杀场景。试想一下，几百来件的商品，在几十万的观众甚至是几百万的观众下，进行抢购。不到一秒的时间被抢购一空，系统需要在这一秒内响应那么多的请求，知道他们的先后顺序。不能多卖出一件，也不能少卖出一件。这时候，秒杀系统就根据需求而产生了，独立于电商系统之外。秒杀其实主要解决两个问题，一个是并发读，一个是并发写。并发读的核心优化理念是尽量减少用户到服务端来“读”数据，或者让他们读更少的数据；并发写的处理原则也一样，它要求我们在数据库层面独立出来一个库，做特殊的处理。另外，还要针对秒杀系统做一些保护，针对意料之外的情况设计兜底方案，以防止最坏的情况发生。

秒杀的整体架构可以概括为“稳、准、快”几个关键字。从技术角度上看“稳、准、快”，就对应了我们架构上的高可用、一致性和高性能的要求，我们的专栏也将主要围绕这几个方面来展开，具体如下：

高性能。秒杀涉及大量的并发读和并发写，因此支持高并发访问这点 非常关键。

一致性。秒杀中商品减库存的实现方式同样关键。

高可用。现实中总难免出现一些我们考虑不到的情况，所以要保证系统的高可用和正确性，我们还要设计一个 PlanB 来兜底，以便在最坏情况发生时仍然能够从容应对。

简单来说，秒杀就是在同一个时刻有大量的请求争抢购买同一个商品并完成交易的过程，用技术的行话来说就是大量的并发读和并发写。

## 二、功能说明

整体框架：

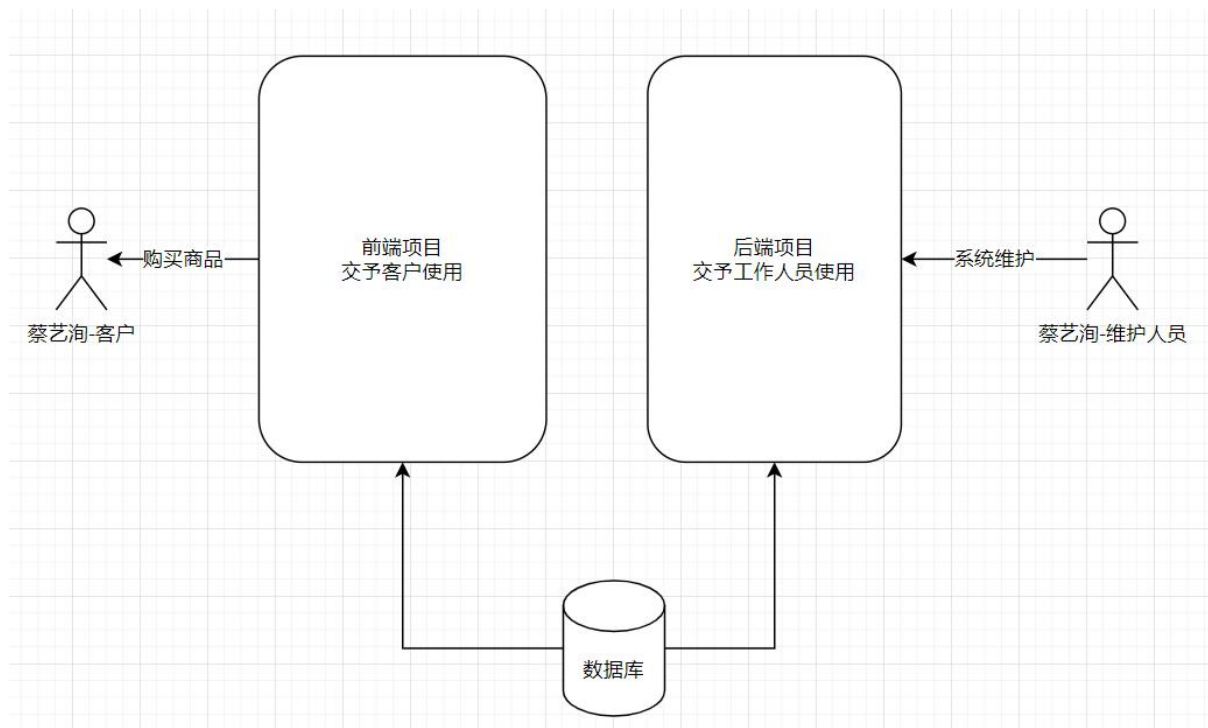


图 1 系统整体逻辑图

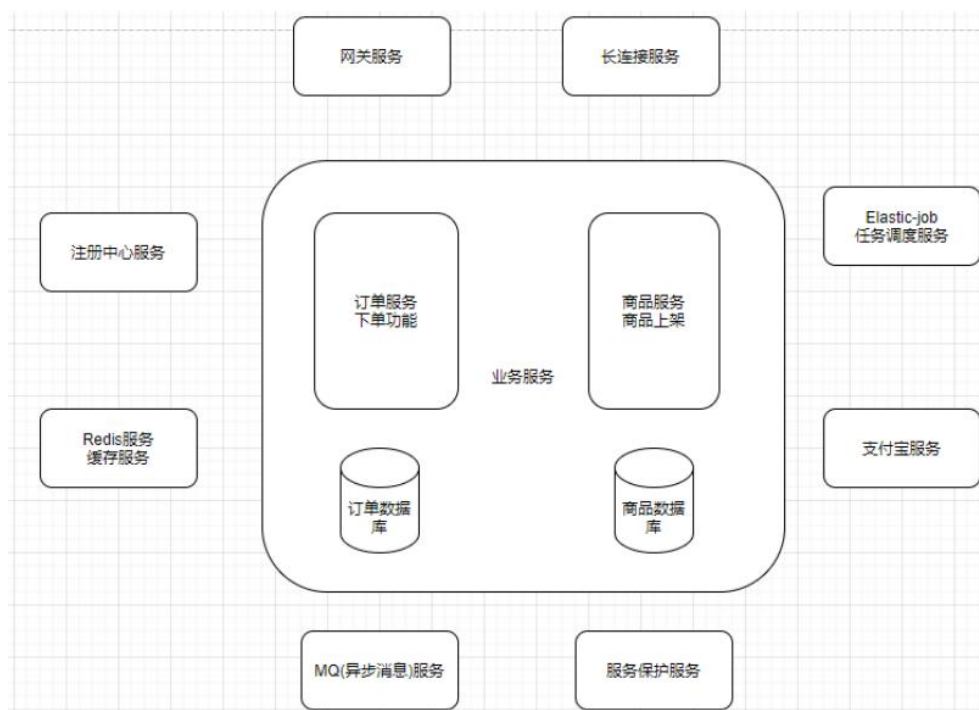


图 2 服务分布图

登录页面设计：

采用 ScketMQ 的生产者、消费者模式。不管登录成功与否，都有日志信息，日志信息的产生和记录就是生产者与消费者模式，异步进行。

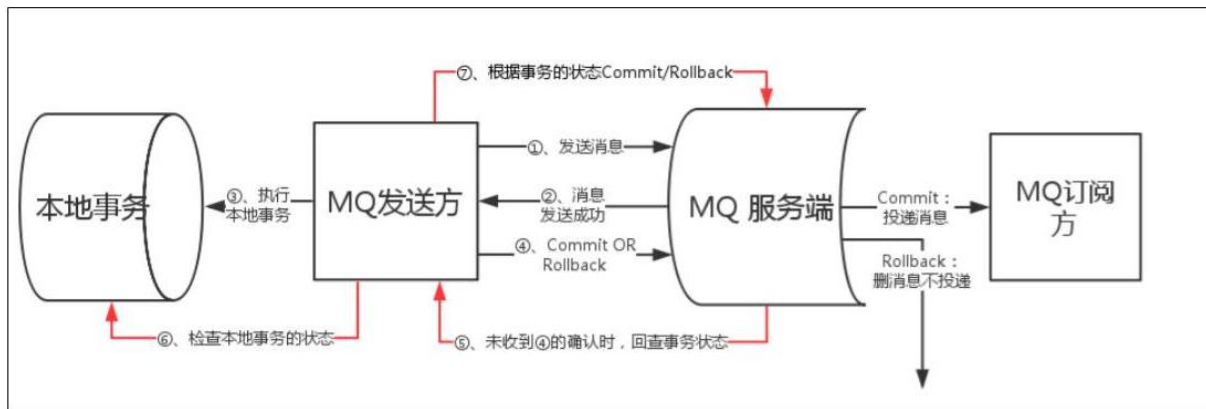


图3 ScoketMQ 事务进行详解

代码托管模式：

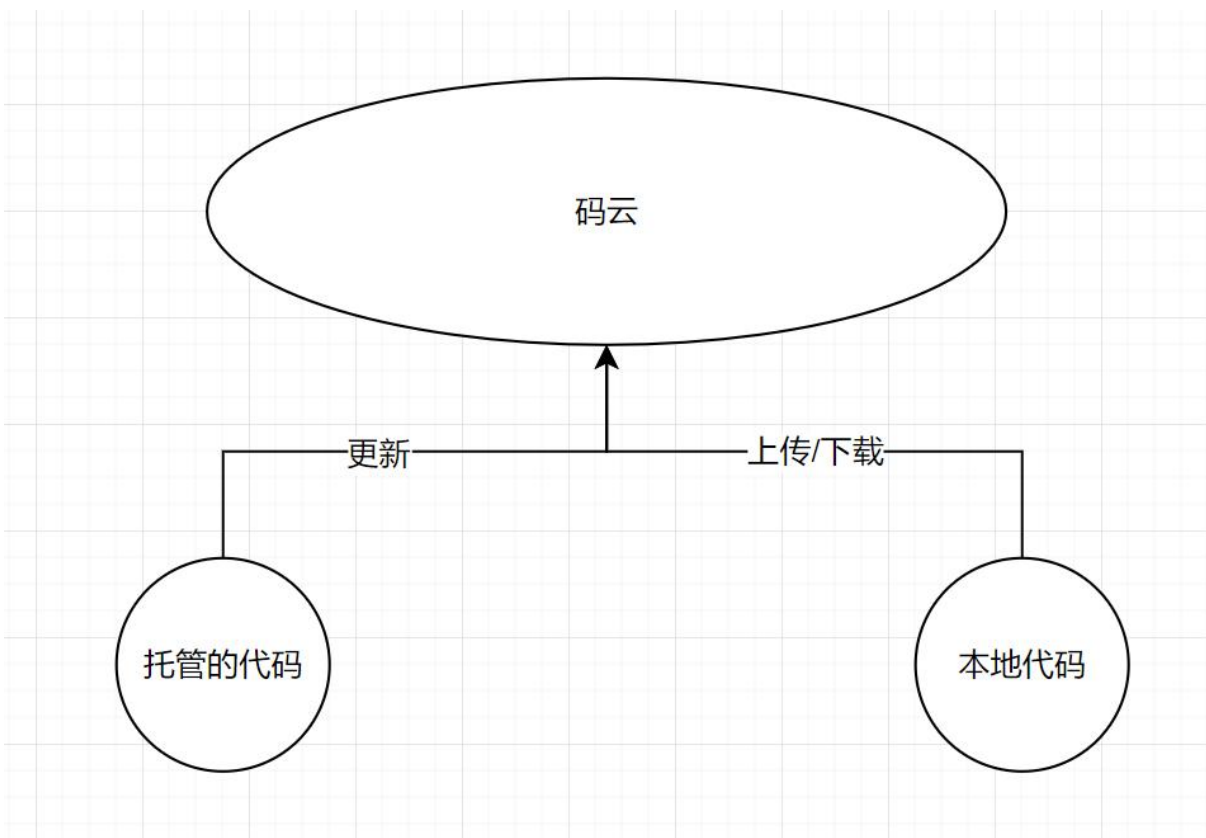


图4 码云的使用

代码使用码云进行托管，可以随时更新代码的进度，上传代码以及下载代码，适合团队的多人协作共同开发一个项目。

数据库的设计:

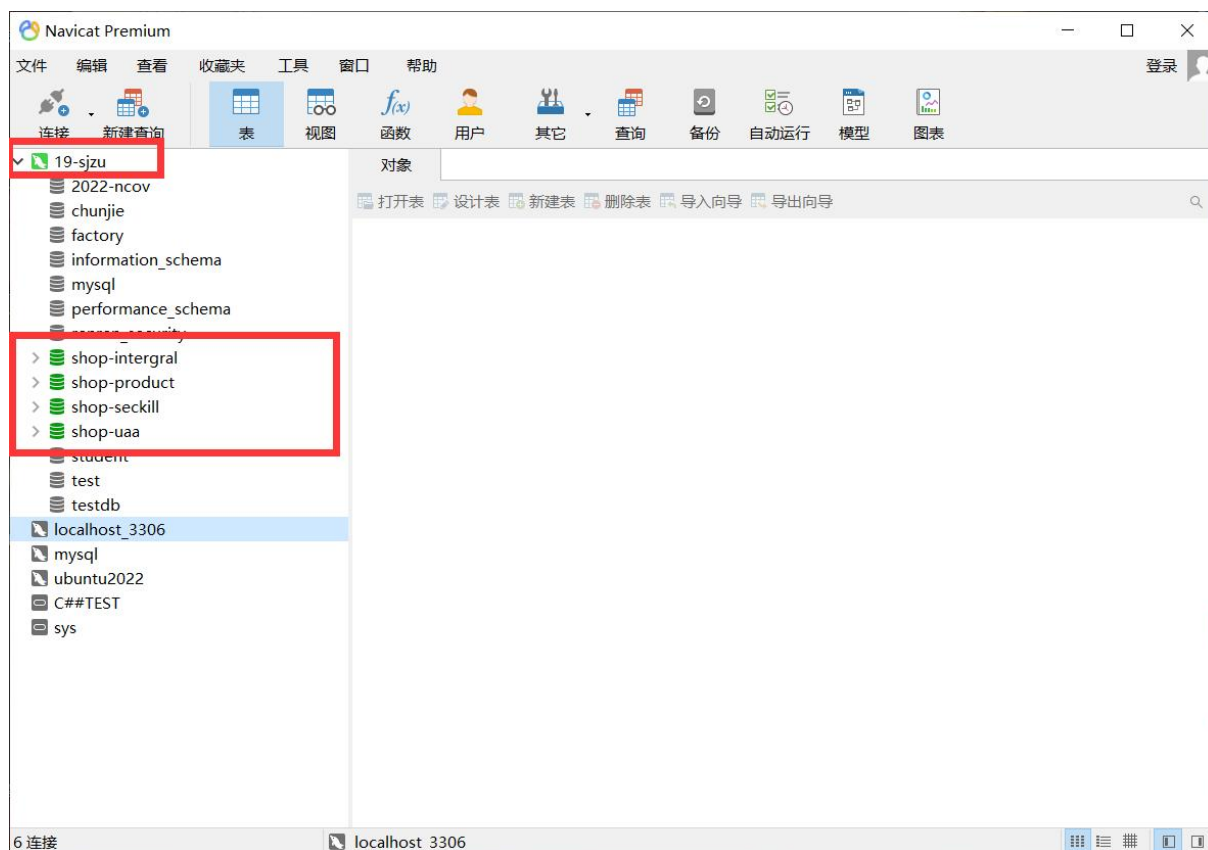


图 5 数据库截图

因为这是一个大项目，而这次综合课设主要是完成这个大项目中的秒杀服务，所以数据库中表格不少，但是有不少表格都是空表。

有四个数据库，分别是 shop-intergral、shop-product、shop-seckill、shop-uaa。

shop-product 主要是存储商品的信息

建立的语句为:

```
CREATE TABLE `t_product` (  
    `id` bigint(20) NOT NULL AUTO_INCREMENT,  
    `product_name` varchar(200) DEFAULT NULL,  
    `product_title` varchar(200) DEFAULT NULL,  
    `product_img` varchar(200) DEFAULT NULL,  
    `product_detail` longtext,  
    `product_price` decimal(10,2) DEFAULT NULL,  
    PRIMARY KEY (`id`)  
) ENGINE=InnoDB AUTO_INCREMENT=43 DEFAULT CHARSET=utf8;
```

存储秒杀信息的数据库 shop-seckill。里面有五个表格，分别是 t\_order\_info、t\_pay\_log、t\_refund\_log、t\_seckill\_product、undo\_log。其中其他表格不做介绍，本次课设没有涉及到。主要是 t\_seckill\_product 这个表格，存储的是秒杀商品的一些信息以及开始秒杀时间。创建语句如下：

```
CREATE TABLE `t_seckill_product` (  
  `id` bigint(20) NOT NULL AUTO_INCREMENT,  
  `product_id` bigint(20) DEFAULT NULL,  
  `seckill_price` decimal(10,2) DEFAULT NULL,  
  `integral` decimal(10,0) DEFAULT NULL,  
  `stock_count` int(255) DEFAULT NULL,  
  `start_date` date DEFAULT NULL,  
  `time` int(11) DEFAULT NULL,  
  PRIMARY KEY (`id`)  
) ENGINE=InnoDB AUTO_INCREMENT=22 DEFAULT CHARSET=utf8;
```

shop-uaa 数据库下有 t\_login\_log、t\_user\_info、t\_user\_login。这个数据库存储的信息主要都是和用户以及登录有关的。其中 t\_user\_login 表记录了用户的电话，也就是用户的账户，还有密码，存储的是密文，以及盐值，用来做 MD5 加密运算的。

t\_user\_info 存储的是用户的一些个人信息，例如电话、名字、头像、ip 地址、注册时间、出生日期、个性签名。

t\_login\_log 表记录的是登录的日志，无论登录成功与否，都会记录下来。包括序列号、电话、ip 地址、登录时间、是否登录成功。

本次项目的秒杀服务涉及的数据库为两个，分别是 shop-product 数据库的 t\_product 表，还有 shop-seckill 数据库的 t\_seckill\_product。

而登录逻辑主要涉及到的数据库是 shop-uaa 数据库下的三个表格。

Redis 的使用：

Redis 的优势：

- 一、性能极高 - Redis 能读的速度是 110000 次/s,写的速度是 81000 次/s 。
- 二、丰富的数据类型 - Redis 支持二进制案例的 Strings, Lists, Hashes, Sets 及 Ordered Sets 数据类型操作。
- 三、原子性 - Redis 的所有操作都是原子性的，意思就是要么成功执行要么失败完全不执行。单个操作是原子性的。多个操作也支持事务，即原子性，通过 MULTI 和 EXEC 指令包起来。
- 四、丰富的特性 - Redis 还支持 publish/subscribe, 通知, key 过期等等特性。

本次的项目也是看中它的这些优势，很适合秒杀要求的“快”、“准”的要求，所以使用 riedis 作为数据的缓存部分参与到项目中。

在本次项目中 redis 的作用如下图所示：

在每一次需要查询数据事，会优先查询 redis 内是否有我们需要的数据，如果有，则直接使用 redis 的数据，如果没有，再进行数据的数据查询，同时会将数据库查询的数据更新到 redis，类似于操作系统中的高速缓冲寄存器的作用。提高数据的查询速度。以适应秒杀时的时间要求。

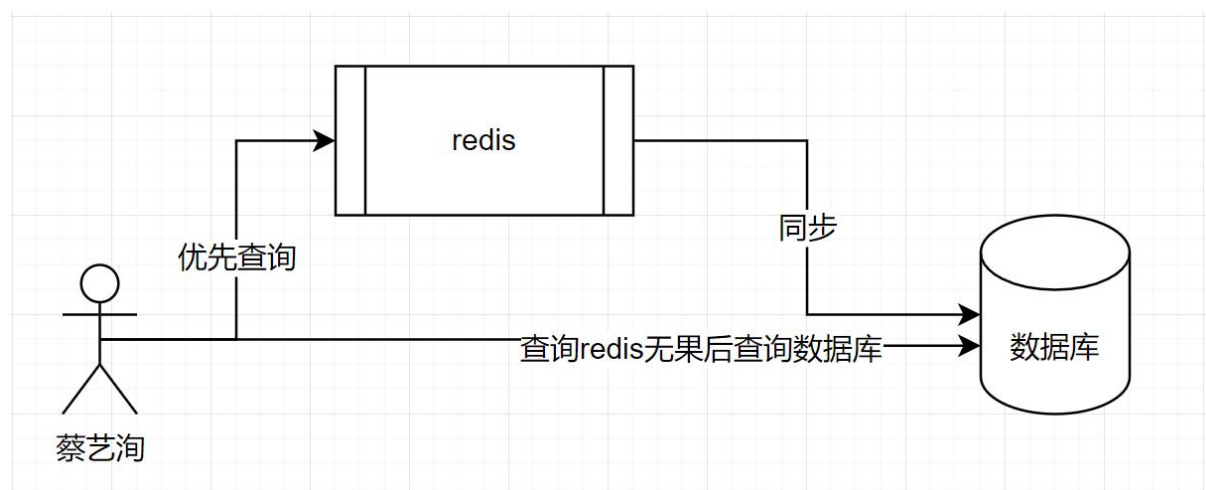


图6 redis 使用的逻辑图



RocketMQ 的使用：  
生产者消费者模式：

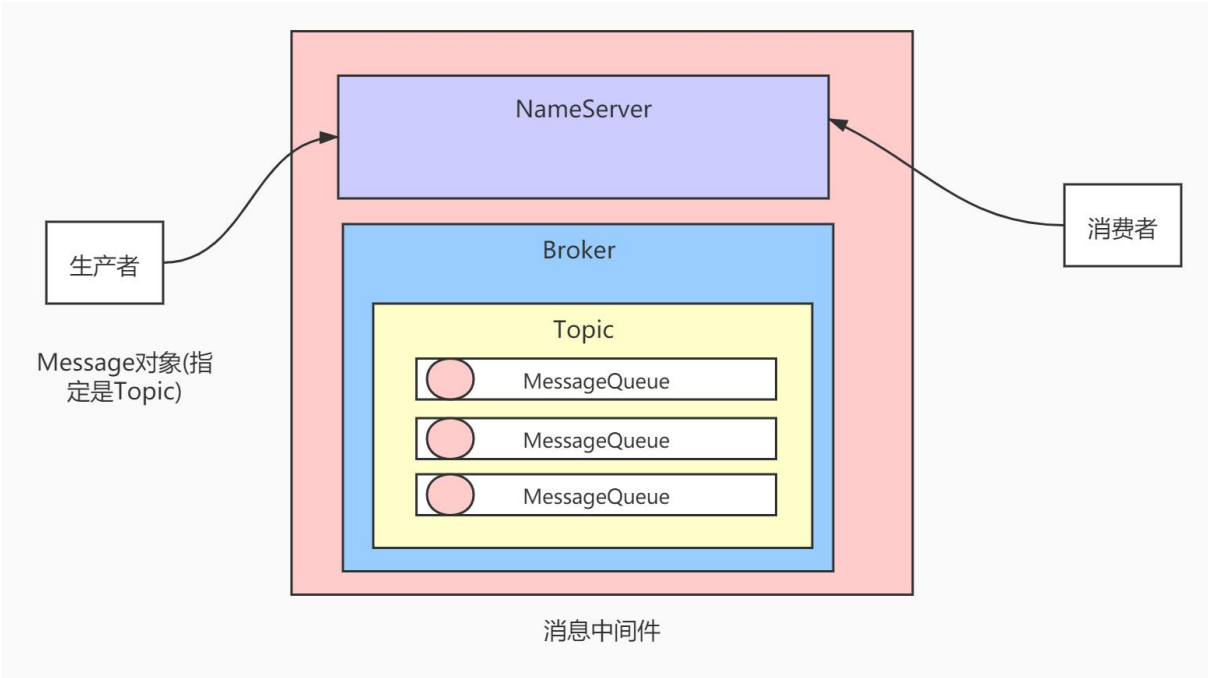


图 7 RocketMQ 组件

RocketMQ 作为一款纯 java、分布式、队列模型的开源消息中间件，支持事务消息、顺序消息、批量消息、定时消息、消息回溯等。它的工作模型简化后如下：

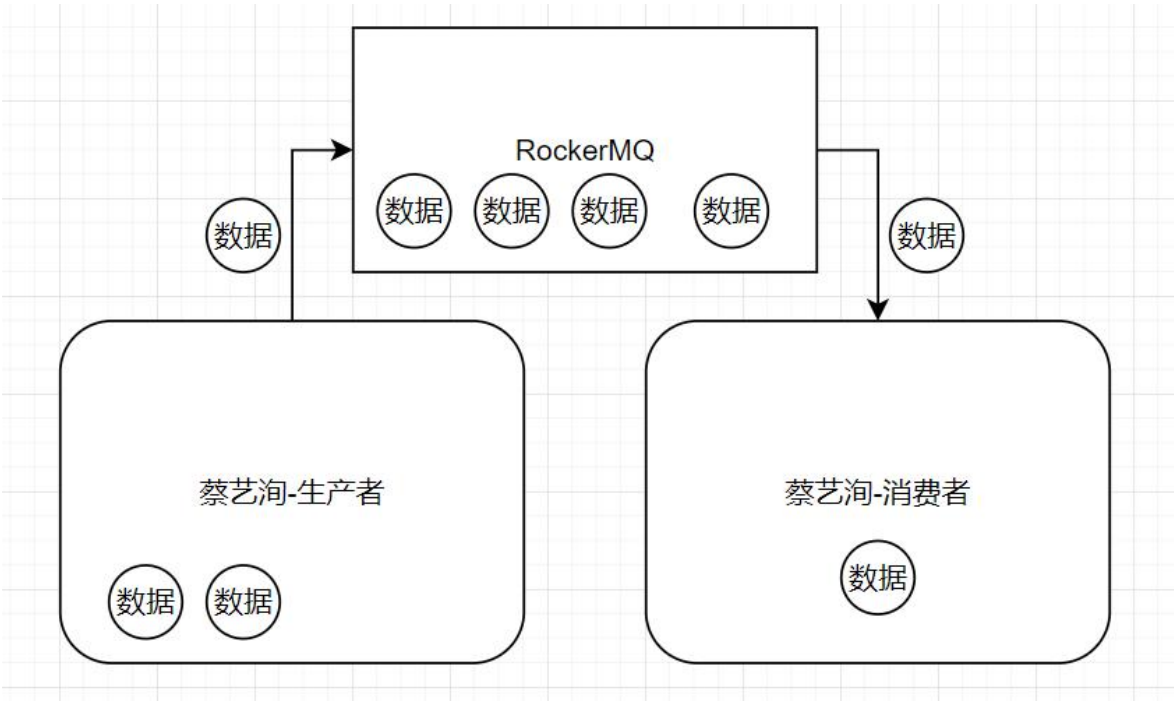


图 8 RocketMQ 模型简化图

这个过程有不同的分工、检验等等工作，但是简化后大体是这样的。至于 RocketMQ 的工作原理是比这个更加细致的，有更多的内容。下面介绍 RocketMQ 的工作原理。

在 RocketMQ 中主要有三个角色，分别是：生产者、RocketMQ、消费者。而 RocketMQ 中又可以分为 name-server 和 Broker 代理服务器，Broker 代理服务器内根据不同的 topic（主题）去划分不同的工作区域，每个工作区域内都有属于自己的 queue（队列）。而 name-server 中有个记录表格，里面记录着不同的 topic 以及相对应的 broker 的信息。

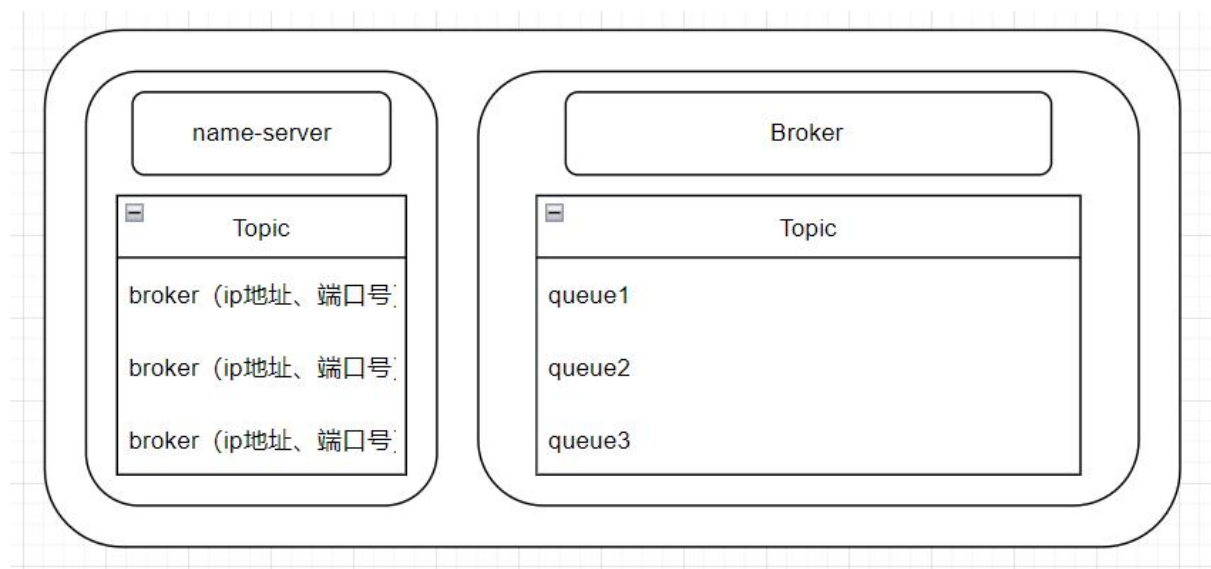


图 9 RocketMQ 的内部

而生产者要做的事是：

- 1.连接 name-server，获取指定主题的元数据
- 2.根据元数据，连接代理服务器，把消息数据发送给指定的队列。

消费者要做的事是：

- 1.连接 name-server，获取指定主题的元数据
- 2.根据元数据找到代理服务器的主题位置
- 3.从主题中某一个队列获取消息数据，返回。

这样一来，登录数据在前端页面产生，后端获取，并写入数据库中，两者可以异步进行。使得逻辑变得简单易用，不容易出错，因为不是秒杀，也没那个时间、先后的需求。

## Nacos 的使用:

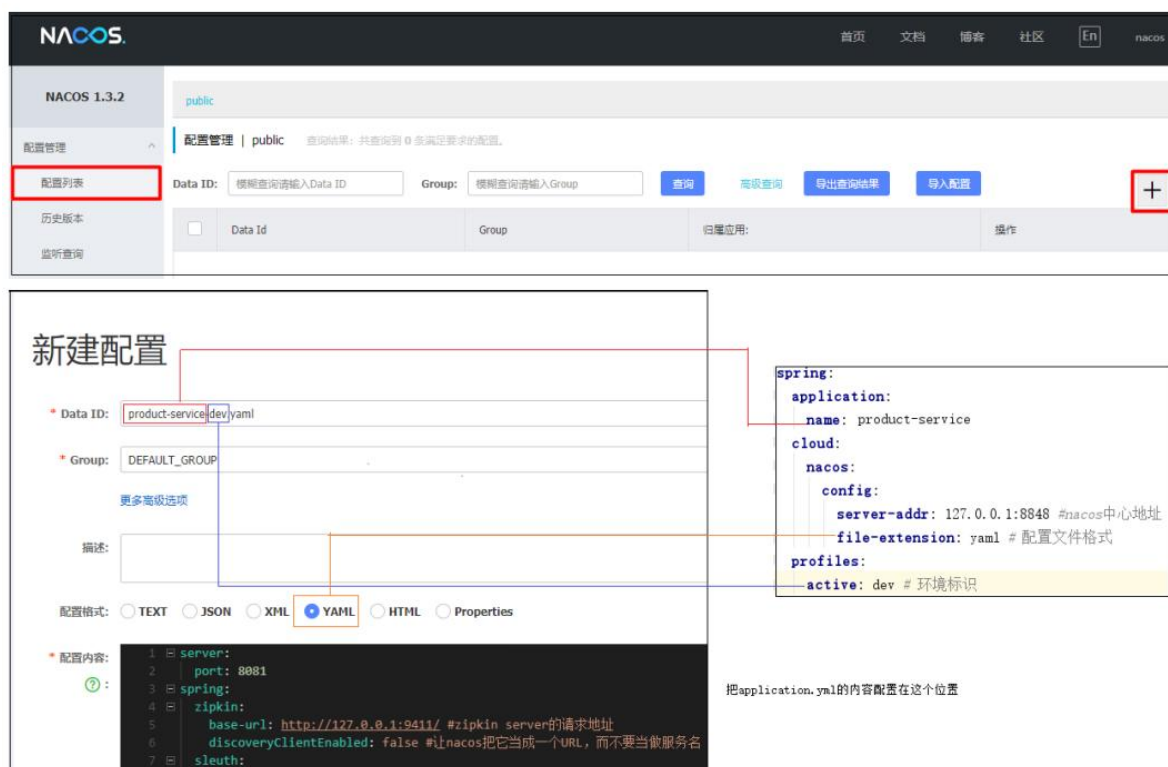


图 10 配置截图

Nacos 的作用就是一个注册中心，用来管理注册上来的各个微服务。因为我们这个项目是使用的微服务，所以需要配置不少的配置。Nacos 就是一个注册管理中心。不同的服务访问 Nacos 注册中心，都会被检测出来，每个服务启动时会把自己注册到注册中心，Nacos 每 5s 心跳一次，15s 都没有收到信息的，服务的元数据处设为 false，表明服务不可用。30s 没有收到服务的信息的，会把这个服务从 map 中删除。当两个服务要相互访问时，会先访问 Nacos，把 map 下载到本地，从里面找到要访问的服务数据，从而进行访问。

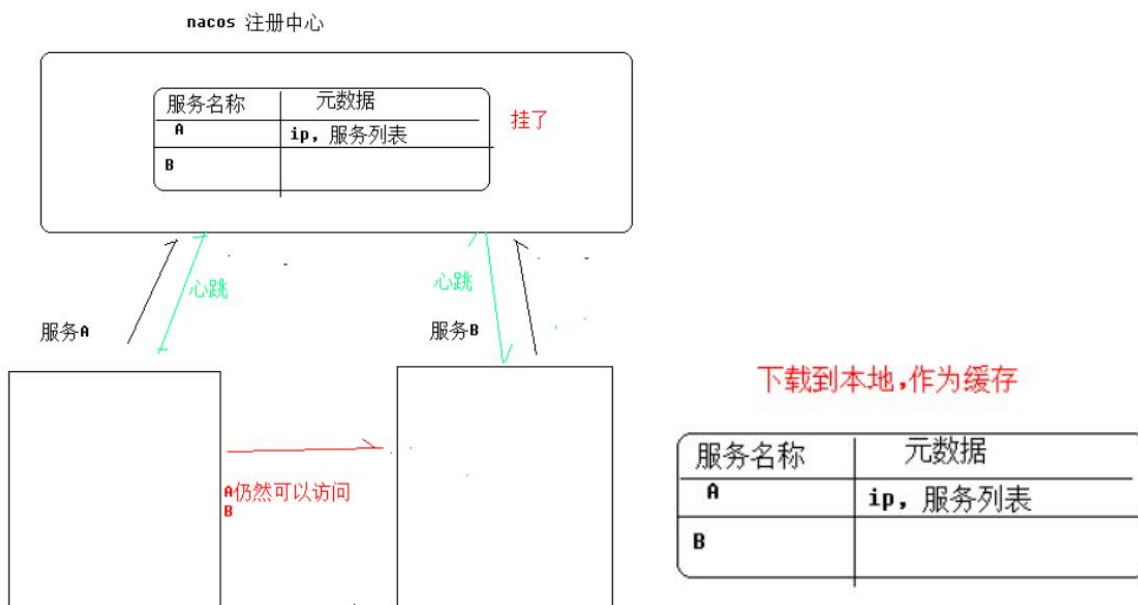
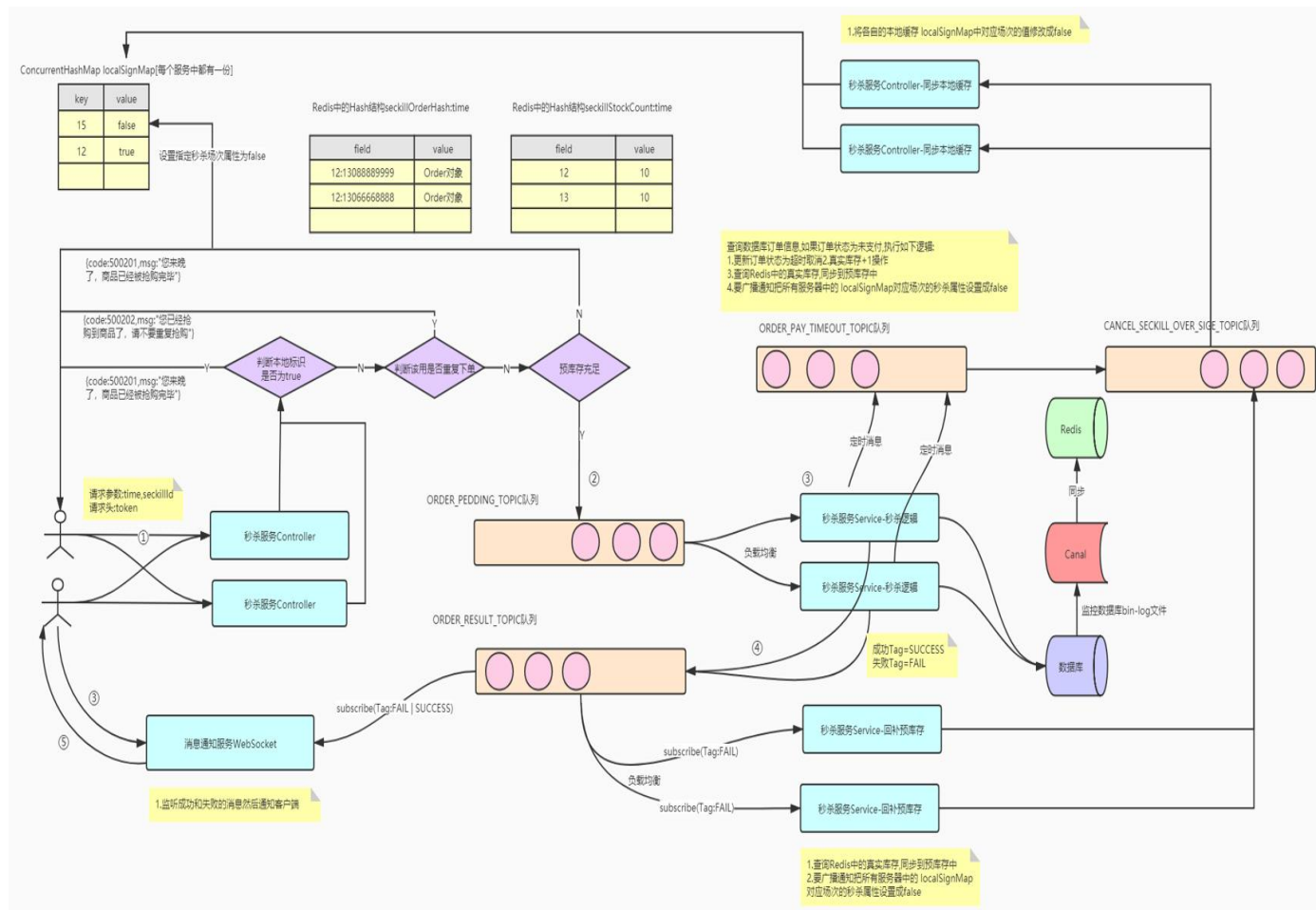


图 11 Nacos 工作流程图

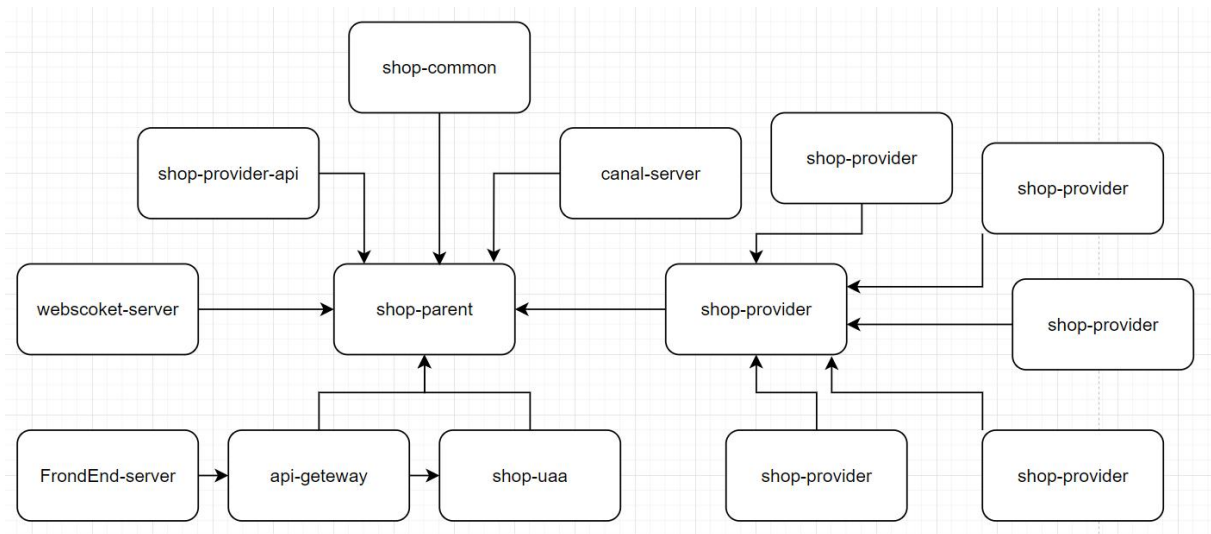
整体程序流程图:



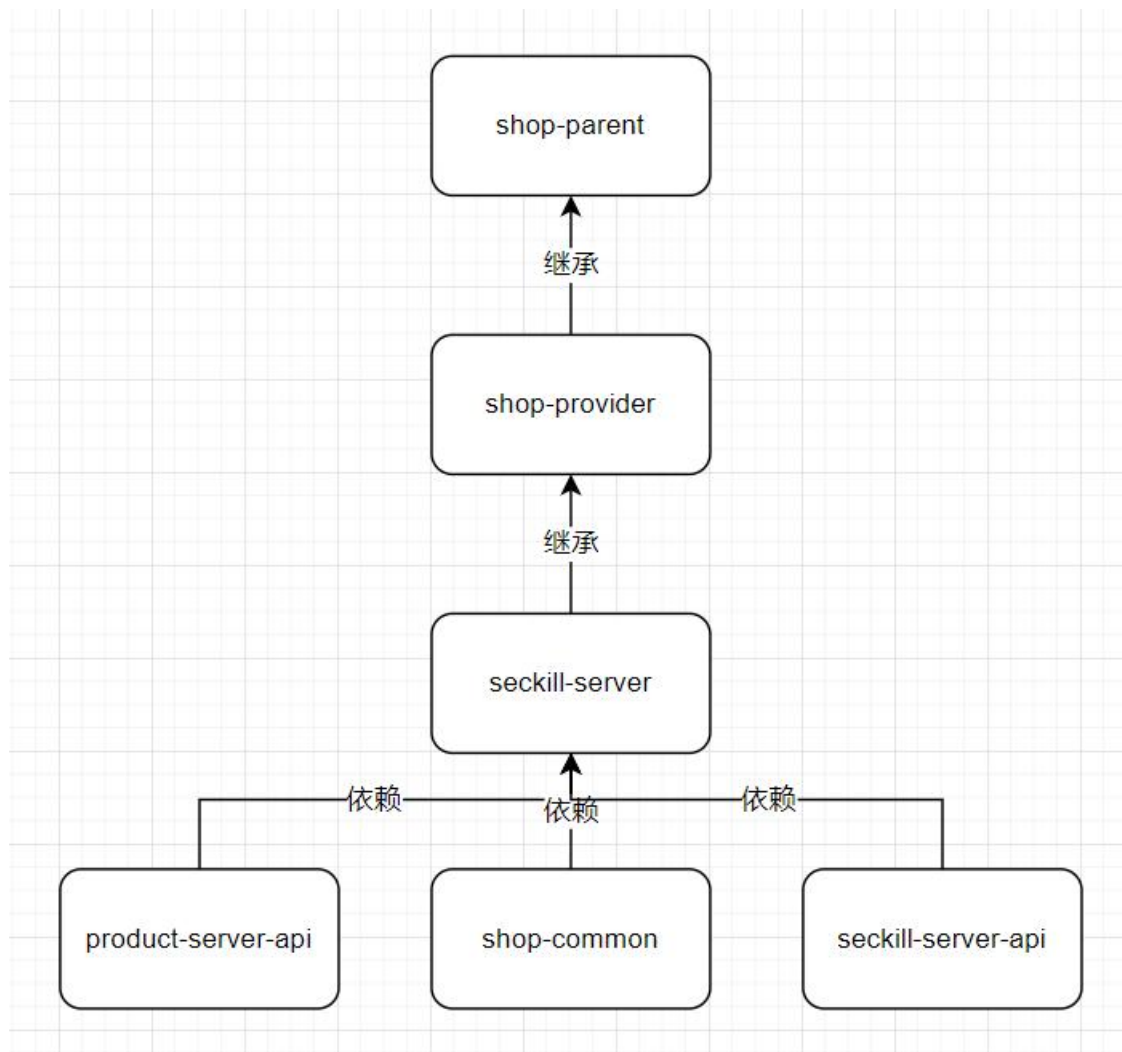
工程结构列表:



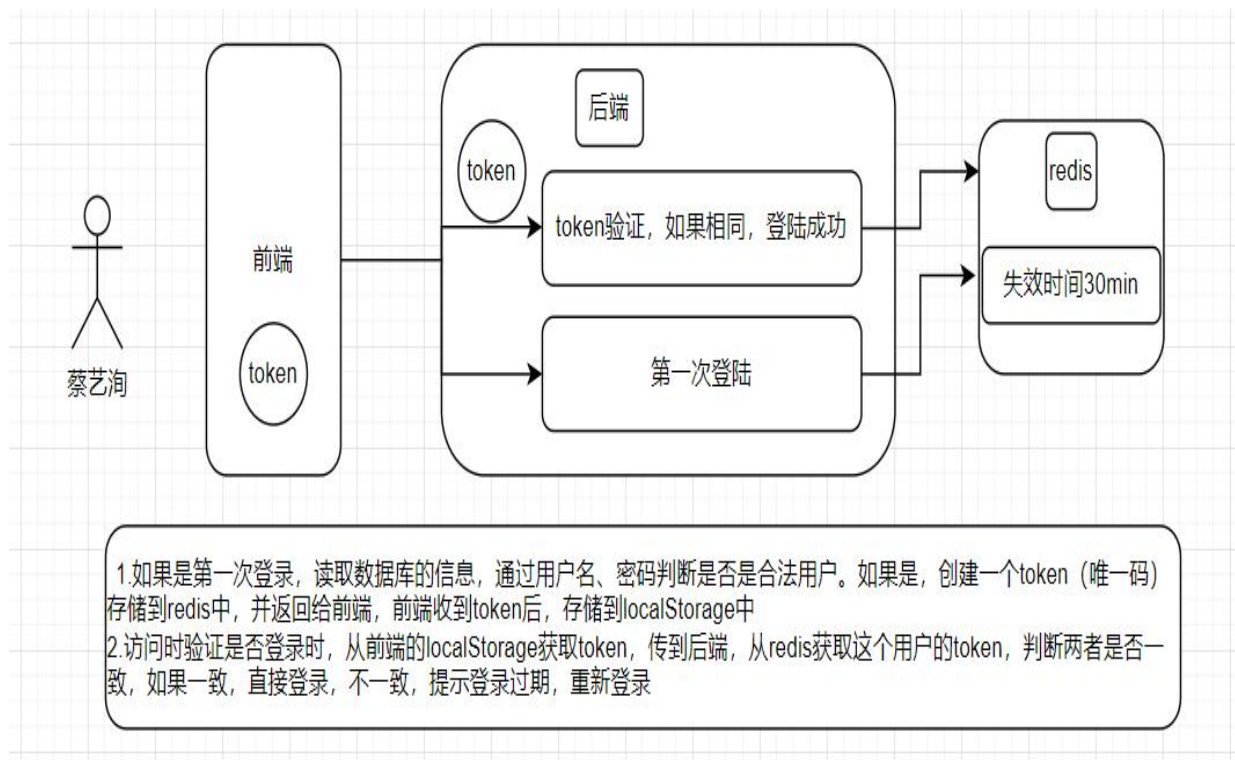
工程的继承关系：



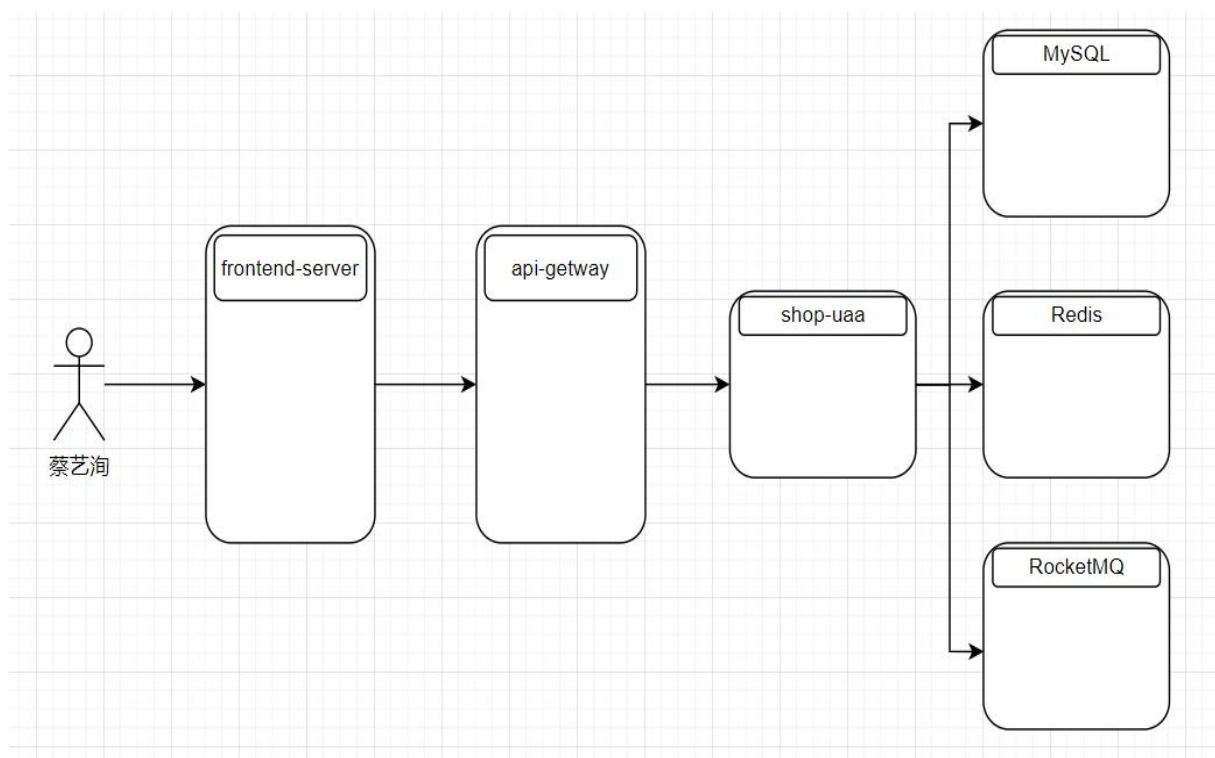
工程的依赖图：



登录的逻辑图：



登录数据的流向：



## 四、程序清单

```
@Service
public class SeckillProductServiceImpl implements ISeckillProductService {
    @Autowired(required = false)
    private SeckillProductMapper seckillProductMapper;
    @Autowired
    private StringRedisTemplate redisTemplate;
    @Autowired
    private RocketMQTemplate rocketMQTemplate;
    @Resource
    private ProductFeignApi productFeignApi;

    @Override
    public List<SeckillProductVo> selectSeckillProduct() {
        //1. 读取秒杀商品表的信息    读取当前的所有秒杀商品信息
        List<SeckillProduct> seckillProductList =
seckillProductMapper.queryAllSeckillProduct();
        //2. 依据商品 id 远程调用产品服务
        List<SeckillProductVo> ret = new ArrayList<>();
        for (SeckillProduct seckillProduct : seckillProductList) {
            Result<Product>
result = productFeignApi.findProductById(seckillProduct.getId());
            if (StringUtils.isEmpty(result) || result.hasError()) {
                throw new BusinessException(CommonCodeMsg.RESULT_ERROR);
            }
            SeckillProductVo vo = new SeckillProductVo();
            Product product = result.getData();
            //3 合并两个表的信息，形成 SeckillProductVo 对象，填充到
List<SeckillProductVo>中
            BeanUtils.copyProperties(product, vo);
            BeanUtils.copyProperties(seckillProduct, vo);
            //System.out.println(vo.getProductDetail());
            ret.add(vo);
        }
        return ret;
    }

    @Override
    public List<SeckillProductVo> queryByTime(Integer time) {
        //从 reids 中获取数据
        String bigSeckillProductKey =
SeckillRedisKey.SECKILL_PRODUCT_HASH.getRealKey(String.valueOf(time));
        //根据 key 或者一组数据
```



```

        Map<Object,Object>
entries=redisTemplate.opsForHash().entries(bigSeckillProductKey);
        //把 map 中的 value 数据移动到 ret 对象中
        List<SeckillProductVo> ret=new ArrayList<>();
        for(Object value:entries.values()){
            String json=(String)value;
            SeckillProductVo seckillProductVo=
JSON.parseObject(json,SeckillProductVo.class);
            ret.add(seckillProductVo);
        }
        return ret;
    }
}

```

```

@Override
public SeckillProductVo find(Integer time, Long seckillId) {
    //从 redis 中获取对应的数据返回
    String bigSeckillProductKey =
SeckillRedisKey.SECKILL_PRODUCT_HASH.getRealKey(String.valueOf(time));
    String smallKey=String.valueOf(seckillId);
    String json=(String)
redisTemplate.opsForHash().get(bigSeckillProductKey,smallKey);
    SeckillProductVo
seckillProductVo=JSON.parseObject(json,SeckillProductVo.class);
    return seckillProductVo;
}
}

```

控制台的类，不同访问之间的调用都要通过这个控制类。

```

@RestController
@RequestMapping("/seckillProduct")
@Slf4j
public class SeckillProductController {
    @Autowired
    private ISeckillProductService seckillProductService;
    @RequestMapping("/selectSeckillProduct")
    public Result<List<SeckillProductVo>> selectSeckillProduct(){
        return Result.success(seckillProductService.selectSeckillProduct());
    }
    @RequestMapping("/queryByTime")
    public Result<List<SeckillProductVo>> queryByTime(Integer time){
        if(StringUtils.isEmpty(time)){
            throw new BusinessException(CommonCodeMsg.PARAM_ERROR);
        }
        return Result.success(seckillProductService.queryByTime(time));
    }
}

```



```

    }
    @RequestMapping("/find")
    public Result<SeckillProductVo> find(Integer time,Long seckillId){
        if(StringUtils.isEmpty(time)||StringUtils.isEmpty(seckillId)){
            throw new BusinessException(CommonCodeMsg.PARAM_ERROR);
        }
        return Result.success(seckillProductService.find(time,seckillId));
    }
}

```

任务的初始化:

```

@Configuration
public class BusinessJobConfig {
    @Bean(initMethod = "init")
    public SpringJobScheduler initUserCacheJob(CoordinatorRegistryCenter registryCenter,
        UserCacheJob userCacheJob){
        LiteJobConfiguration jobConfiguration =
        ElasticJobUtil.createDefaultSimpleJobConfiguration(userCacheJob.getClass(),
        userCacheJob.getCron());
        SpringJobScheduler springJobScheduler = new SpringJobScheduler(userCacheJob,
        registryCenter,jobConfiguration );
        return springJobScheduler;
    }
    @Bean(initMethod = "init")
    public SpringJobScheduler initSeckillSyncJob(CoordinatorRegistryCenter registryCenter,
        SeckillSeckillSyncJob seckillSeckillSyncJob){
        LiteJobConfiguration jobConfiguration =
        ElasticJobUtil.createDefaultSimpleJobConfiguration(seckillSeckillSyncJob.getClass(),
        seckillSeckillSyncJob.getCron());
        SpringJobScheduler springJobScheduler =
        new SpringJobScheduler(seckillSeckillSyncJob, registryCenter,jobConfiguration );
        return springJobScheduler;
    }
}

```

定时秒杀任务的发布:

Component

@Setter@Getters

@RefreshScope

@Slf4j

public class SeckillSeckillSyncJob implements SimpleJob {

@Value("\${jobCron.initSeckillProduct}")

private String cron;

@Autowired

private StringRedisTemplate redisTemplate;

@Autowired

```

private SeckillProductFeignApi seckillProductFeignApi;
@Override
public void execute(ShardingContext shardingContext) {
    doWork();
}
private void doWork() {
    //删除 redis 中秒杀商品的所有信息有两个秒杀商品信息，秒 杀商品的库存信
    息
    int[] times={10,12,14};
    for(int time:times){
        String                bigSeckillProductKey                =
SeckillRedisKey.SECKILL_PRODUCT_HASH.getRealKey(String.valueOf(time));
        String
bigSecillProductStoreKey=SeckillRedisKey.SECKILL_STOCK_COUNT_HASH.getRealKe
y(String.valueOf(time));
        redisTemplate.delete(bigSeckillProductKey);
        redisTemplate.delete(bigSecillProductStoreKey);
    }
    //        读取秒杀服务中的所有秒杀商品                远程服务调用
    Result<List<SeckillProductVo>>                result=
seckillProductFeignApi.selectSeckillProduct();
    if(StringUtils.isEmpty(result)||result.hasError()){
        throw new BusinessException(CommonCodeMsg.RESULT_ERROR);
    }
    for(SeckillProductVo seckillProductVo:result.getData()){
        String                bigSeckillProductKey                =
SeckillRedisKey.SECKILL_PRODUCT_HASH.getRealKey(String.valueOf(seckillProductVo.
getTime()));
        String
bigSecillProductStoreKey=SeckillRedisKey.SECKILL_STOCK_COUNT_HASH.getRealKe
y(String.valueOf(seckillProductVo.getTime()));
        //秒杀商品信息

        redisTemplate.opsForHash().put(bigSeckillProductKey,String.valueOf(seckillProductVo.getId
()), JSON.toJSONString(seckillProductVo));
        redisTemplate.opsForHash().put(bigSecillProductStoreKey,
String.valueOf(seckillProductVo.getId()),String.valueOf(seckillProductVo.getStockCount()));
    }

}

商品服务的 Sql 查询语句:
<mapper namespace="cn.wolfcode.mapper.ProductMapper" >
    <select id="queryProductByIds" resultType="cn.wolfcode.domain.Product" >
        select  id,product_name,product_title,product_img,product_detail,product_price  from

```

```

t_product
    <where>
        id in
        <foreach item="item" collection="ids" separator="," open="(" close=")" >
            #{item}
        </foreach>
    </where>
</select>
<select id="queryProductById" resultType="cn.wolfcode.domain.Product">
    select id,product_name,product_title,product_img,product_detail,product_price from
t_product where id=#{productId}
</select>
</mapper>

```

商品秒杀服务的 sql 查询语句:

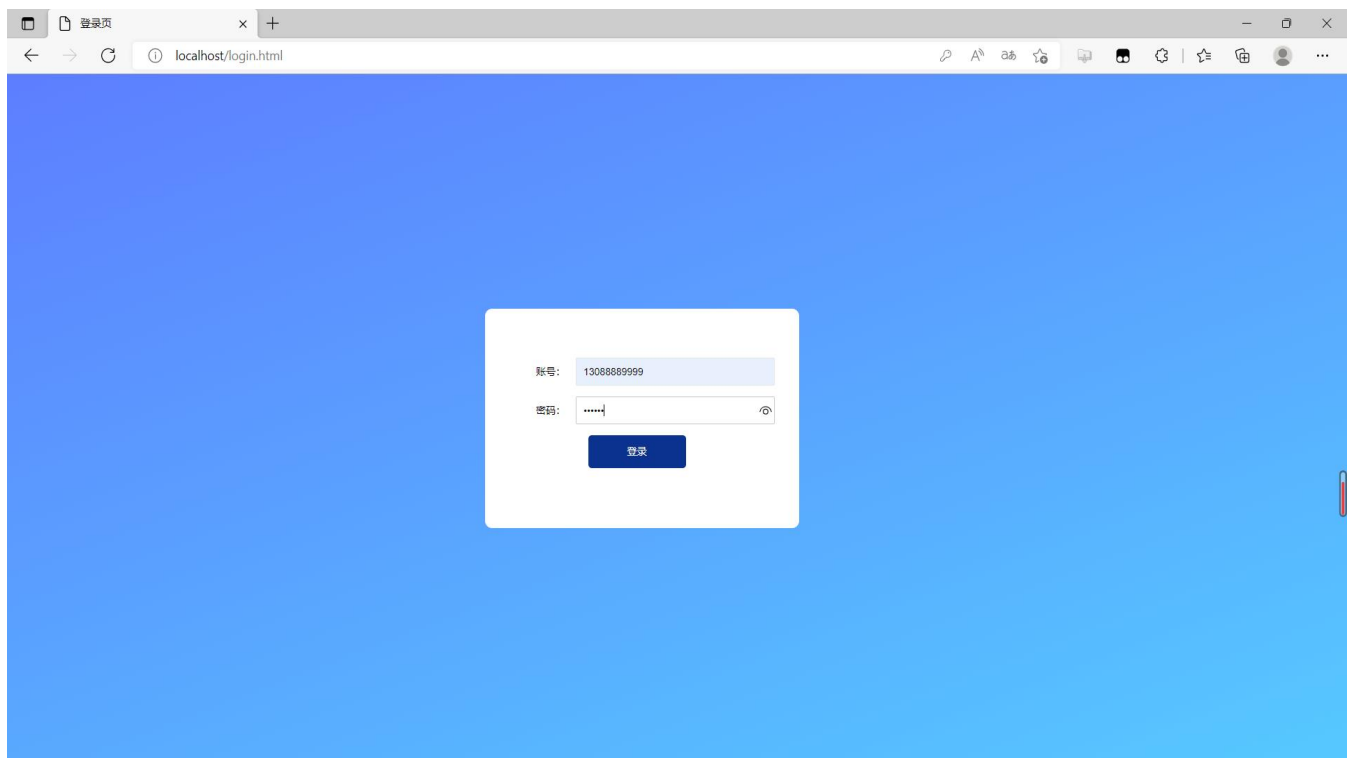
```

<mapper namespace="cn.wolfcode.mapper.SeckillProductMapper" >
    <select id="queryCurrentlySeckillProduct"
resultType="cn.wolfcode.domain.SeckillProduct" >
        select id,product_id,seckill_price,stock_count,time,intergral,start_date from
t_seckill_product where to_days(start_date) = to_days(now()) and time = #{time} ;
    </select>
    <update id="decrStock">
        update t_seckill_product set stock_count = stock_count - 1 where id = #{seckillId} and
stock_count > 0
    </update>
    <update id="incrStock">
        update t_seckill_product set stock_count = stock_count + 1 where id = #{seckillId}
    </update>
    <select id="getStockCount" resultType="int">
        select stock_count from t_seckill_product where id = #{seckillId}
    </select>
    <select id="queryAllSeckillProduct" resultType="cn.wolfcode.domain.SeckillProduct">
        select id,product_id,seckill_price,stock_count,time,intergral,start_date from
t_seckill_product where to_days(start_date) = to_days(now());
    </select>
</mapper>

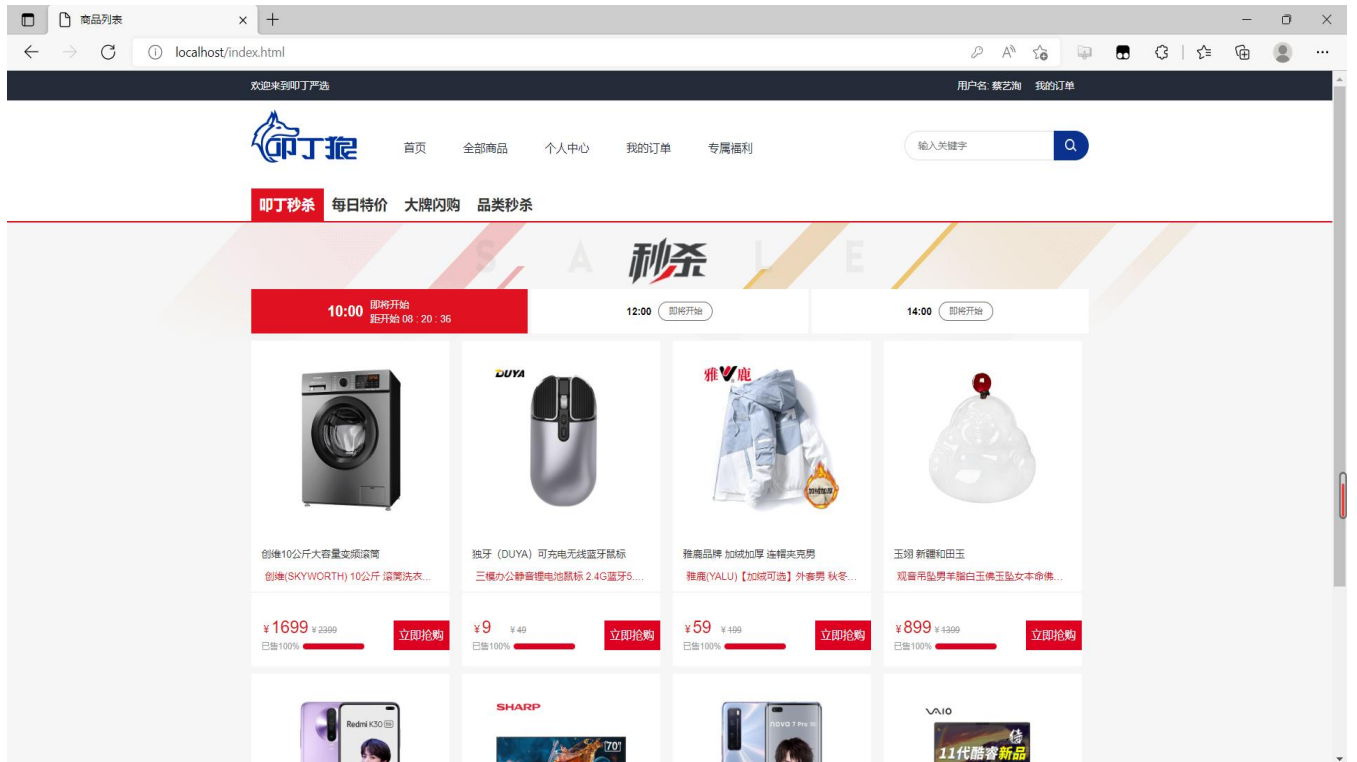
```

## 五、运行结果

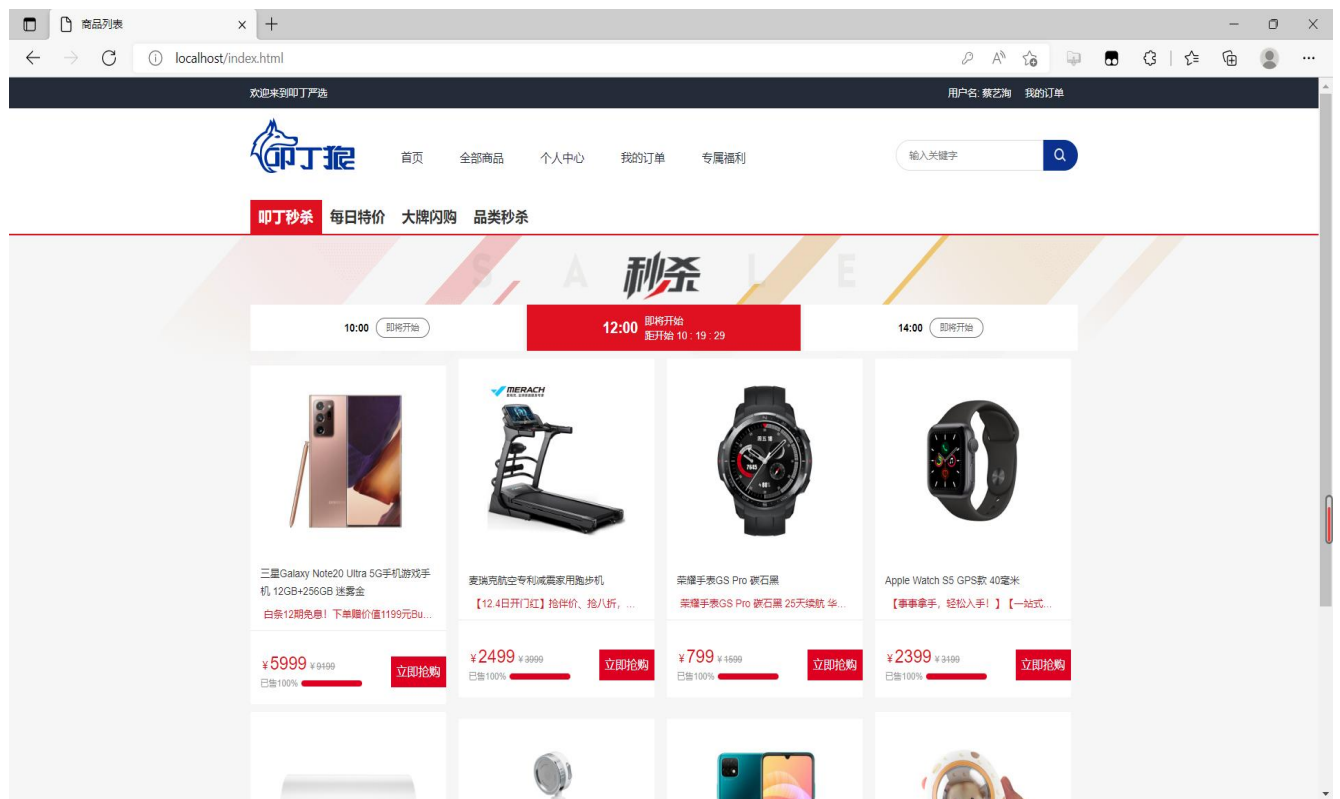
登录页面运行结果：



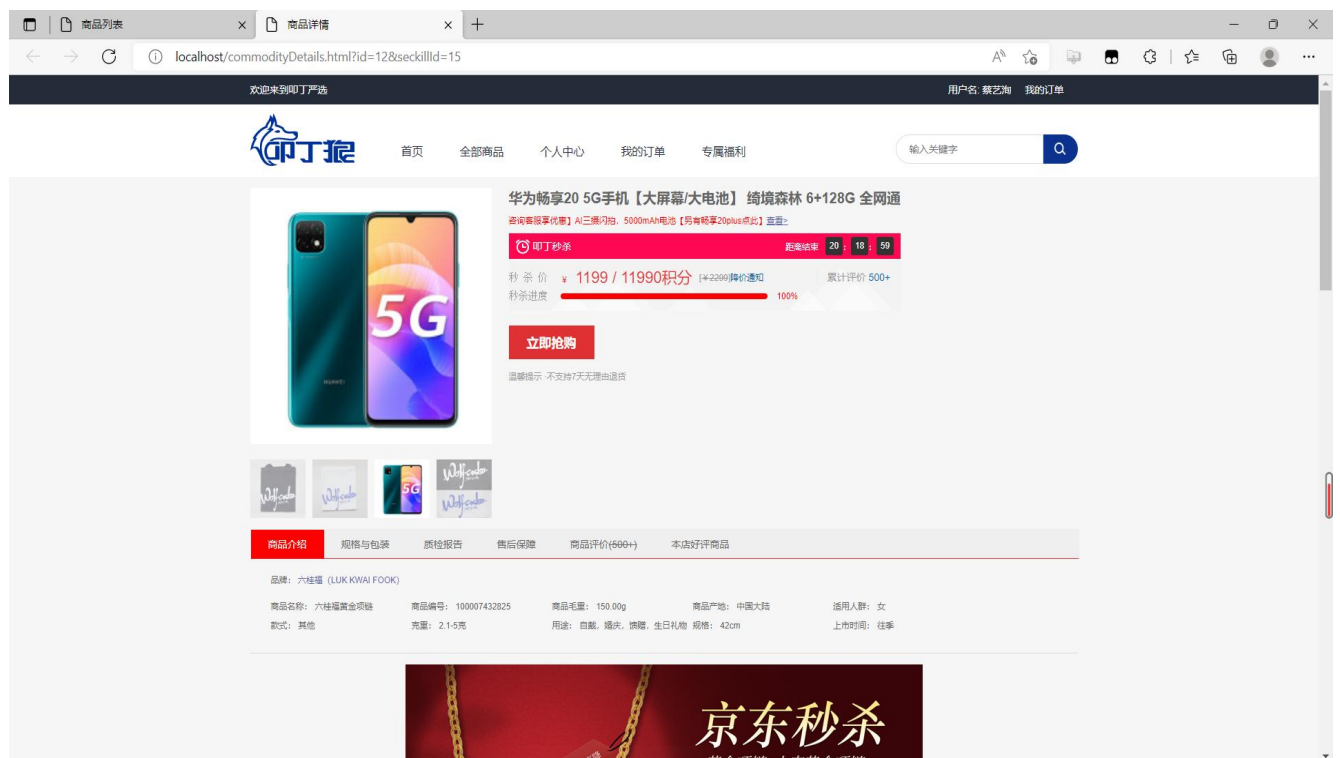
秒杀界面运行结果：



不同时间段的秒杀商品界面运行结果：



秒杀商品的详情界面运行结果：



## 六、设计体会

本次的课程设计，我接触到的是一个比较庞大的项目，尽管我没能窥探到这个项目的全局面貌，因为综合课设的时间太短，课上老师只能选择秒杀部分来教我们，但是如此严谨、复杂的项目结构已经让我知道了什么是企业级开发。

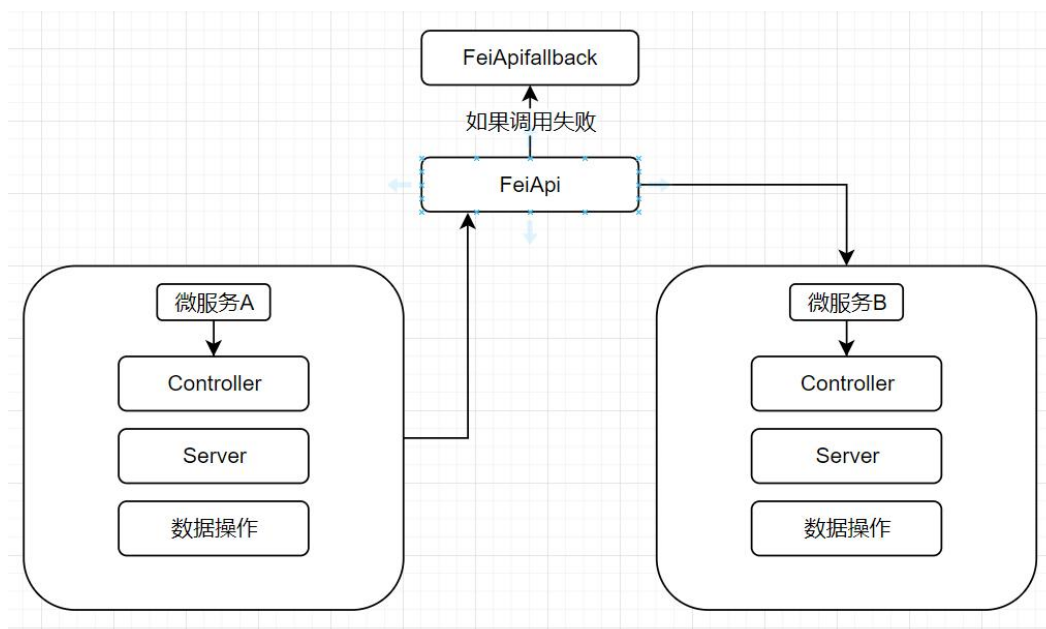
因为上课老师一直都是强调微服务，而且这个项目的基础都是微服务，以及微服务之间的调用。我们学习的部分重点是在秒杀部分，它涉及的微服务有三个，分别是 `job-server`、`product-server`、`seckill-server`，分别是任务微服务、商品微服务、以及秒杀微服务。

项目首先要解决的是两大问题：1.微服务内部应该是什么样的结构？2.微服务之间如何进行调用？

微服务的内部结构，这边有理论上的分层，也有实际上的代码的分层。因为我们都是实操，所以我就讲代码上的分层吧。通过分成 `Controller` 层、`Server` 层、数据操作层。和传统不一样的地方是，`Controller` 层是处理不同微服务之间的调用，来决定使用那种服务，算是一个指挥中心了。而 `Server` 实现的是基础服务，包括一些数据查询的类，登录的类等等，而 `Server` 层里面会调用数据查询方法，也就是数据操作层。不一样的地方是，数据查询语句，它没有专门建立一个数据库连接池，而是将数据库连接写在了 `Nacos` 上，而数据查询语句写在了本地的 `xml` 文件里面。

而微服务之间的调用方法，都是通过 `Fei` 包下的 `FeiApi` 类来实现微服务之间的调用，但是都要经过 `Controller` 类来总体的调度。

`@FeignClient(name="product-service",fallback = ProductFallback.class)` 就是调用 `product-server` 服务，如果不成功，`ProductFallback.class` 就是用来兜底的类。而 `@RequestMapping("/product/findProductById")` 则指定了数据查询调用哪种方法。这个方法一定是写在 `product-server` 服务的 `Controller` 类里面的。不然就会使用兜底的那个类里面的方法，其实就是返回一个空值。



总的来说，这次的综合课程设计，我学到了很多的东西，特别是这种企业级开发的框架，因为这是一个成熟体系的框架，任何地方的细节大多已经被完善了，而且为了适应不同的特殊场景而对设计层面进行特殊的改造，都有很多地方让我花比这次课设周更长的时间去学习，而不会仅仅停留在课设周去学习，而且那样短的时间学成也不现实。

## 七、参考文献

- [1] Spring Cloud Alibaba-pdf 资料
- [2] 微服务之-ServiceMesh
- [3] [https://zhuanlan.zhihu.com/p/149693905?from\\_voters\\_page=true](https://zhuanlan.zhihu.com/p/149693905?from_voters_page=true)
- [4] RocketMQ 官方示例-pdf 资料
- [5] RocketMQ 核心架构.pdf 资料

## 八、教师评价